
Arquitectura ELT en Databricks para análisis comercial y abastecimiento de inventario

Metodología Medallion

Identificación del Trabajo

Asignatura: Big Data

Institución: Universidad Autónoma Latinoamericana (UNAULA)

Elaborado por:

Raúl Fernando Ossa Ramírez CC: 71.993.011

raul.ossa3011@unaula.edu.co

Jennifer Alejandra López Sánchez CC: 1020411872

jennifer.lopez1872@unaula.edu.co

1. Introducción y Metodología

Este proyecto presenta una solución de ingeniería de datos orientada a optimizar el abastecimiento de inventario mediante el análisis de movimientos comerciales históricos, como ventas y devoluciones. Utilizando Databricks y Apache Spark, se desarrolló un pipeline ELT basado en la arquitectura Medallion (Bronze, Silver y Gold), permitiendo la ingesta, depuración, transformación y análisis de datos para la generación de reportes y modelos de sugerido de compras.

ARQUITECTURA MEDALLION – SUGERIDO DE COMPRAS

Pipeline de datos en Databricks para análisis de ventas, devoluciones y sugerido de compras

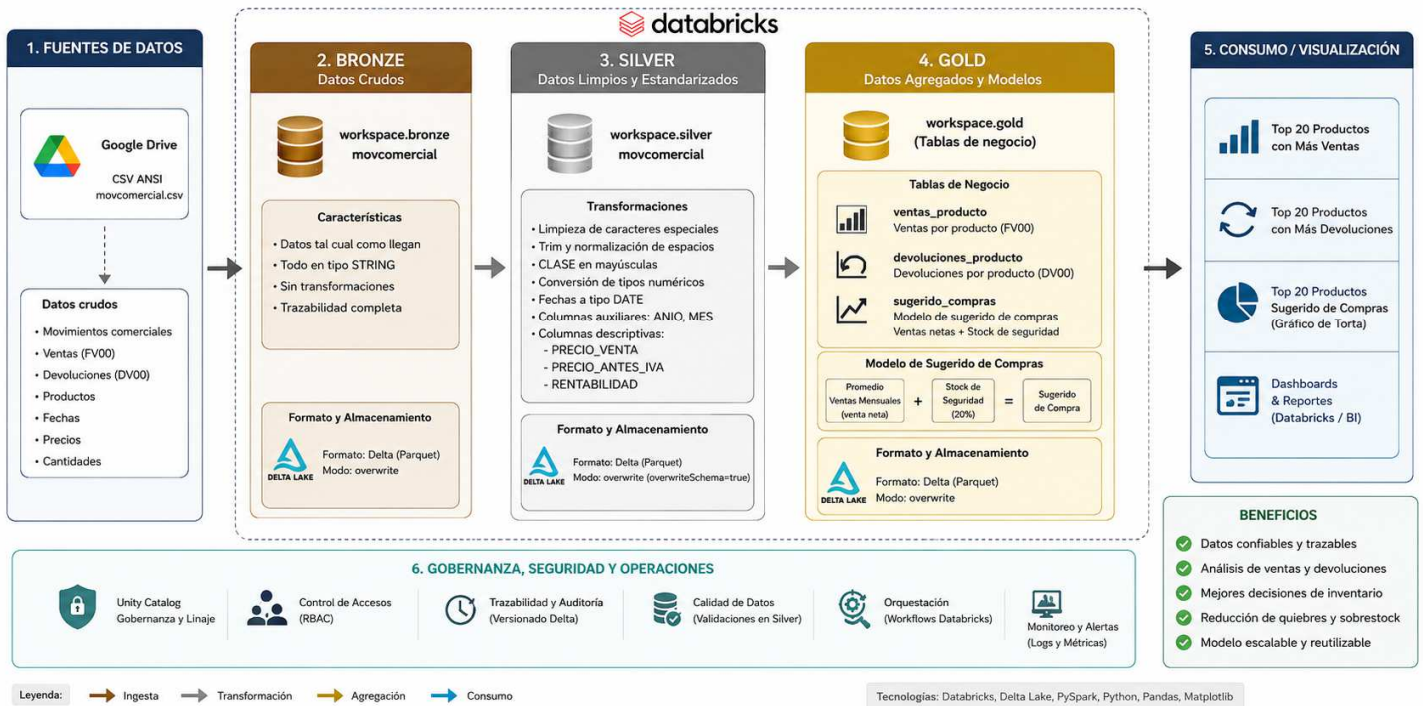


Figura 1: Arquitectura de Datos de Extremo a Extremo

El flujo comienza con la ingesta de archivos planos desde fuentes externas, transitando por procesos de limpieza profunda y normalización, hasta culminar en un modelo analítico que predice el volumen óptimo de compras mensuales, considerando tanto la demanda real como las devoluciones históricas.

Workspace > Users > nandoossa@gmail.com >

Trabajo_Final_BD feature-pruebas Open in editor ☆

Search Type Owner Last modified

Name	Type	Owner
pipelines	Folder	nandoossa@gmail.com
01_ingestar_datos_bronze	Notebook	nandoossa@gmail.com
02_silver	Notebook	nandoossa@gmail.com
03_gold	Notebook	nandoossa@gmail.com
04_modelo_compras	Notebook	nandoossa@gmail.com
DOCUMENTACION.pdf	File	nandoossa@gmail.com
JOB_INGESTA.png	File	nandoossa@gmail.com
MEDALLION MAP.png	File	nandoossa@gmail.com
PIPELINE.png	File	nandoossa@gmail.com
README.md	File	nandoossa@gmail.com

Figura 2: Espacio de trabajo Databricks

2. Capa Bronze: Ingesta de datos

El Notebook **01_ingestar_datos_bronze** es el punto de entrada. Su propósito es capturar los datos crudos sin alterar su estructura original, garantizando la trazabilidad histórica. En esta etapa se definen los esquemas captura la información en formato Delta.

Proceso de ingesta:

Este notebook realiza la ingesta de datos en un entorno de Databricks siguiendo estos pasos:

- Preparación: Crea las bases de datos para las capas Bronze, Silver y Gold.
- Extracción: Lee un archivo CSV de movimientos comerciales (usando ; como separador) desde una ubicación en Google Worspace mediante URL.
- Limpieza Inicial: Convierte la codificación de los datos a UTF-8 para evitar errores de caracteres.
- Estandarización: Define todas las columnas como texto (string) para asegurar que la carga no falle por tipos de datos incorrectos.
- Carga (Bronze): Guarda los datos resultantes en la tabla workspace.bronze.movcomercial usando el formato Delta (basado en Parquet).

Código utilizado:

```
%sql
-- Crear el esquema para la capa Bronze CREATE
SCHEMA IF NOT EXISTS workspace.bronze;

-- Crear el esquema para la capa Silver CREATE
SCHEMA IF NOT EXISTS workspace.silver;

-- Crear el esquema para la capa Gold CREATE
SCHEMA IF NOT EXISTS workspace.gold; import
pandas as pd
from pyspark.sql.types import ( StructType,
    StructField,
    StringType
)
# ===== #
ID DEL ARCHIVO EN GOOGLE
# =====
file_id = "184dVBWTbhJ6Um64Pgmd0364kQvvDQHv0" #
===== #
CONSTRUIR URL DIRECTA
# =====
url = f"https://drive.google.com/uc?id={file_id}" #
=====
# LEER CSV ANSI (cp1252)
# =====
```

```

pdf = pd.read_csv(
    url,
    sep=";",
    encoding="cp1252",
    dtype=str,
    low_memory=False
)

# =====
# REEMPLAZAR NULOS
# =====
pdf = pdf.fillna("")
# =====
# CREAR ESQUEMA TODO STRING
# (Recomendado para capa Bronze)
# =====
schema = StructType([
    StructField(col, StringType(), True)
    for col in pdf.columns
])

# =====
# CREAR DATAFRAME SPARK
# =====
df = spark.createDataFrame(
    pdf.to_dict(orient="records"),
    schema=schema
)

# =====
# MOSTRAR INFORMACIÓN
# =====
display(df)
df.printSchema()
print(f"Total registros: {df.count()}")

# =====
# CREAR ESQUEMA BRONZE SI NO EXISTE
# =====
spark.sql("""
CREATE SCHEMA IF NOT EXISTS workspace.bronze
""")

# =====
# GUARDAR TABLA EN FORMATO DELTA
# (Internamente usa archivos parquet)
# =====
df.write \
    .format("delta") \
    .mode("overwrite") \

```

```
.saveAsTable("workspace.bronze.movcomercial")

print("Tabla workspace.bronze.movcomercial creada correctamente")

# =====
# VALIDAR TABLA CREADA
# =====
df_bronze = spark.table("workspace.bronze.movcomercial")
display(df_bronze)

# =====
# VER DETALLE DEL STORAGE
# =====
spark.sql("""
DESCRIBE DETAIL workspace.bronze.movcomercial
""").display()
```

3. Capa Silver: Ingeniería de Datos, Limpieza y Depuración

En el Notebook **02_silver**, se aplica la mayor carga de procesamiento técnico. Se transforman los datos crudos de la capa Bronze en un conjunto de datos confiable y ordenado.

| Transformaciones Críticas realizadas:

- Configuración: Define rutas de acceso entre las carpetas bronze y silver.
- Carga: Lee las tablas crudas de Clientes, Productos y Ventas.
- Limpieza: Elimina nulos, borra duplicados y corrige formatos (fechas y números).
- Guardado: Exporta los datos limpios a la capa Silver en formato Delta.
- En resumen: Convierte los datos crudos de la primera etapa en datos confiables y tipificados.

Código utilizado en Databricks:

```
from pyspark.sql.functions import (
    col,
    trim,
    regexp_replace,
    to_date,
    year,
    month,
    when,
    expr,
    upper
)

# =====
# LEER TABLA DESDE BRONZE
# =====
df = spark.table("workspace.bronze.movcomercial")

# =====
# LIMPIAR CARACTERES ESPECIALES Y ESPACIOS
# =====
for column in df.columns:

    df = df.withColumn(
        column,
        trim(
            regexp_replace(
```

```

        regexp_replace(col(column), r'[\n\r\t]', ' '),
        r'\s+',
        ' '
    )
)
)

# =====
# NORMALIZAR CLASE
# =====
df = df.withColumn(
    "CLASE",
    upper(trim(col("CLASE")))
)

# =====
# REEMPLAZAR VACÍOS POR NULL
# =====
for column in df.columns:

    df = df.withColumn(
        column,
        when(col(column) == "", None)
        .otherwise(col(column))
    )

# =====
# COLUMNAS NUMÉRICAS
# =====
numeric_columns = [
    "UNIDAD",
    "CANTIDAD",
    "PARCIAL",
    "PARCIALANT",
    "COSTO",
    "PARLANTIVA",
    "UTILIDAD",
    "RENTABILID"
]

# =====
# LIMPIAR Y CONVERTIR COLUMNAS NUMÉRICAS
# =====
for column in numeric_columns:

    # Eliminar separador de miles "."
    df = df.withColumn(
        column,
        regexp_replace(col(column), r"\.", "")
    )

```

```

# Reemplazar coma decimal "," por "."
df = df.withColumn(
    column,
    regexp_replace(col(column), ",", ".")
)

# Eliminar caracteres no numéricos
df = df.withColumn(
    column,
    regexp_replace(col(column), r"^[^0-9\.-]", "")
)

# Convertir a DOUBLE
df = df.withColumn(
    column,
    expr(f"try_cast({column} as double)")
)

# =====
# CREAR COLUMNAS MÁS DESCRIPTIVAS
# =====
df = df.withColumn(
    "PRECIO_VENTA",
    col("PARCIAL")
)

df = df.withColumn(
    "PRECIO_ANTES_IVA",
    col("PARCIALANT")
)

df = df.withColumn(
    "RENTABILIDAD",
    col("RENTABILID")
)

# =====
# CONVERTIR FECHA
# =====
df = df.withColumn(
    "FECHA",
    to_date(col("FECHA"), "yyyy-MM-dd HH:mm:ss")
)

# =====
# CREAR COLUMNAS AUXILIARES
# =====
df = df.withColumn(
    "ANIO",
    year(col("FECHA"))
)

```

```

)

df = df.withColumn(
    "MES",
    month(col("FECHA"))
)

# =====
# MOSTRAR RESULTADO
# =====
display(df)
df.printSchema()
print(f"Total registros: {df.count()}")

# =====
# CREAR ESQUEMA SILVER SI NO EXISTE
# =====
spark.sql("""
CREATE SCHEMA IF NOT EXISTS workspace.silver
""")

# =====
# GUARDAR TABLA SILVER
# =====
df.write \
    .format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable("workspace.silver.movcomercial")
print("Tabla workspace.silver.movcomercial creada correctamente")

# =====
# VALIDAR TABLA
# =====
df_silver = spark.table("workspace.silver.movcomercial")
display(df_silver)

# =====
# VER DETALLE
# =====
spark.sql("""
DESCRIBE DETAIL workspace.silver.movcomercial
""").display()

```

Además, se realizó la normalización de la columna CLASE, eliminando espacios en blanco y forzando mayúsculas, lo cual es vital para diferenciar correctamente entre Facturas (FV00) y Devoluciones (DV00).

4. Capa Gold: Análisis del Negocio

El Notebook **03_gold** consume los datos ya saneados para generar agregaciones que permiten visualizar el comportamiento comercial de los productos.

Análisis de Desempeño Comercial General:

- **Ventas:** Consolidación de ingresos y unidades por producto.
- **Devoluciones:** Identificación de mermas y logística inversa por referencia.
- **Visualización:** Uso de librerías como `Matplotlib` para generar rankings de los productos más relevantes.

```
from pyspark.sql.functions import (
    col,
    upper,
    trim,
    abs,
    first,
    regexp_replace,
    expr,
    round,
    countDistinct,
    sum,
    avg
)

# =====
# LEER DATOS DESDE SILVER
# =====
df = spark.table("workspace.silver.movcomercial")

# =====
# NORMALIZAR CLASE
# =====
df = df.withColumn(
    "CLASE",
    upper(trim(col("CLASE")))
)
```

```

# =====
# LIMPIAR Y CONVERTIR CANTIDAD
# =====
df = df.withColumn(
    "CANTIDAD_TMP",
    regexp_replace(col("CANTIDAD"), r"\.", "")
)
df = df.withColumn(
    "CANTIDAD_TMP",
    regexp_replace(col("CANTIDAD_TMP"), ",", ".")
)
df = df.withColumn(
    "CANTIDAD_TMP",
    expr("try_cast(CANTIDAD_TMP as double)")
)

# =====
# FILTRAR SOLO VENTAS
# =====
df_ventas = df.filter(
    col("CLASE") == "FV00"
)

# =====
# VENTAS POR PRODUCTO
# =====
ventas_producto = df_ventas.groupBy(
    "PRODUCTO"
).agg(

    first("PRODUCTONO")
    .alias("NOMBRE_PRODUCTO"),
    # CONTAR FACTURAS DE VENTA
    countDistinct("NUMERO")
    .alias("TOTAL_VENTAS"),
    # SUMAR UNIDADES VENDIDAS
    round(
        abs(sum("CANTIDAD_TMP")),
        2
    ).alias("UNIDADES_VENDIDAS"),
    # PRECIO TOTAL DE VENTA
    round(
        sum("PARCIAL"),
        2
    ).alias("PRECIO_VENTA"),
    # PRECIO TOTAL ANTES DE IVA
    round(
        sum("PARCIALANT"),
        2
    )
)

```

```

        ).alias("PRECIO_ANTES_IVA"),
        # RENTABILIDAD %
        round(
            avg("RENTABILIDAD") * 100,
            2
        ).alias("RENTABILIDAD_PORCENTAJE")

    ).orderBy(
        col("TOTAL_VENTAS").desc()
    )

# =====
# MOSTRAR RESULTADO
# =====
display(ventas_producto) from pyspark.sql.functions import (
    col,
    upper,
    trim,
    abs,
    first,
    regexp_replace,
    expr,
    round,
    countDistinct,
    sum,
    avg
)

import matplotlib.pyplot as plt
# =====
# LEER DATOS DESDE SILVER
# =====
df = spark.table("workspace.silver.movcomercial")

# =====
# NORMALIZAR CLASE
# =====
df = df.withColumn(
    "CLASE",
    upper(trim(col("CLASE")))
)

# =====
# LIMPIAR Y CONVERTIR CANTIDAD
# =====
df = df.withColumn(
    "CANTIDAD_TMP",
    regexp_replace(col("CANTIDAD"), r"\.", "")
)

```

```

df = df.withColumn(
    "CANTIDAD_TMP",
    regexp_replace(col("CANTIDAD_TMP"), ",", ".")
)
df = df.withColumn(
    "CANTIDAD_TMP",
    expr("try_cast(CANTIDAD_TMP as double)")
)

# =====
# FILTRAR SOLO VENTAS
# =====
df_ventas = df.filter(
    col("CLASE") == "FV00"
)

# =====
# VENTAS POR PRODUCTO
# =====
ventas_producto = df_ventas.groupBy(
    "PRODUCTO"
).agg(

    first("PRODUCTONO")
    .alias("NOMBRE_PRODUCTO"),
    # CONTAR FACTURAS DE VENTA
    countDistinct("NUMERO")
    .alias("TOTAL_VENTAS"),
    # SUMAR UNIDADES VENDIDAS
    round(
        abs(sum("CANTIDAD_TMP")),
        2
    ).alias("UNIDADES_VENDIDAS"),
    # PRECIO TOTAL DE VENTA
    round(
        sum("PARCIAL"),
        2
    ).alias("PRECIO_VENTA"),
    # PRECIO TOTAL ANTES DE IVA
    round(
        sum("PARCIALANT"),
        2
    ).alias("PRECIO_ANTES_IVA"),
    # RENTABILIDAD %
    round(
        avg("RENTABILIDAD") * 100,
        2
    ).alias("RENTABILIDAD_PORCENTAJE")

).orderBy(

```

```
col("TOTAL_VENTAS").desc()

)

# =====
# TOP 20
# =====

top20 = ventas_producto.limit(20)

# =====
# CONVERTIR A PANDAS
# =====

pdf = top20.toPandas()

# =====
# GRAFICAR
# =====

plt.figure(figsize=(14,8))

plt.bar(
    pdf["NOMBRE_PRODUCTO"],
    pdf["TOTAL_VENTAS"]
)

plt.xticks(rotation=90)

plt.xlabel("Producto")
plt.ylabel("Total Ventas")
plt.title("Top 20 Productos con Más Ventas")

plt.tight_layout()

plt.show()
```

	PRODUCTO	NOMBRE_PRODUCTO	TOTAL_DEVOLUCIONES	UNIDADES_DEVUELTAS	PRECIO_VENTA	PRECIO_ANTES_IVA	RENTABILIDAD_PORCENTAJE
49	601206	DISP AGUA SWAN LB-IWB1.5-5X88R DE NEVERA - SILVER	12	720	41731200	35068235.29	-34.17
50	8012120	T.V. SWAN 70" UHD 4KWEBOS LED SMART	12	170	46257028	38871452.13	-39.53
51	601205	DISP AGUA SWAN LB-IWB15-5X55R DE NEVERA	12	180	10233050	8599201.67	-37.83
52	108086	LAV L.G WT13DPBK.ASFECOL	11	170	28570446	24008778.14	-18
53	3011110	NEV HACEB 220 CE TI R2	11	110	17686587	14862678.13	-31.64
54	901233	VITRINA SWAN RVV 211L MA CS	11	120	24607234	20945348.85	-37.33
55	108090	LAV L.G WT19MYTB.ABMECOL	11	130	24310048	20428611.76	-13.77
56	251279	CONG. SWAN CHS SW1-145	11	180	18106722	15215732.72	-36.61
57	304632	NEV MABE RMP421FBCG	11	110	24226542	20358438.66	-20.36
58	251267	CONG SWAN CHS 512L MA CS	11	110	28237499	23728990.78	-29
59	901219	CONG SWAN CHS 100 MA CS	11	190	12418554	10435759.7	-15.05
60	3011104	NEV HACEB ALC 404 SE DA MI PLOMO R2	10	100	21179381	17797799.17	-11.6
61	1070127	LAV. SAMSUNG WA13CGS441BDCO	10	210	30636068	25744594.91	-11.48
62	251278	CONG. SWAN CVV-460	10	100	63300000	53193277.31	-30.4
63	1512143	PARLANTE SWAN SW-D08 RECARGABLE	10	100	6745570	5668546.23	-45.5

Figura 3: Visualización de los datos en Gold

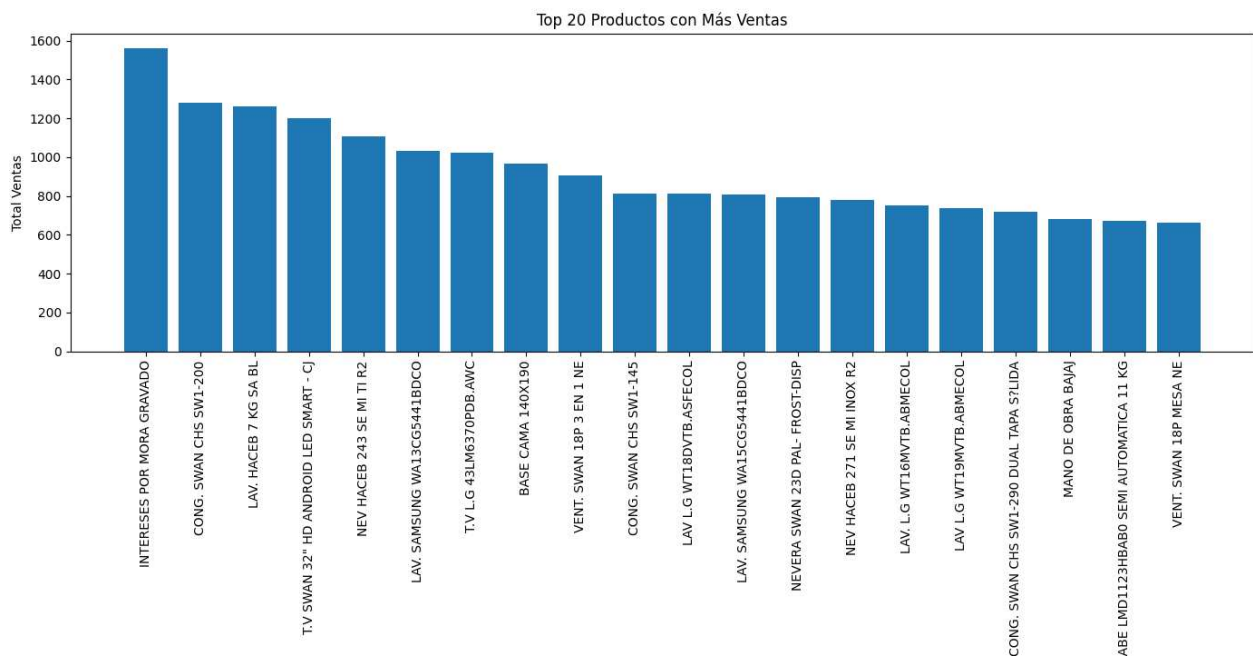


Figura 4: Gráfica del comportamiento de las ventas por Marca – Top 20

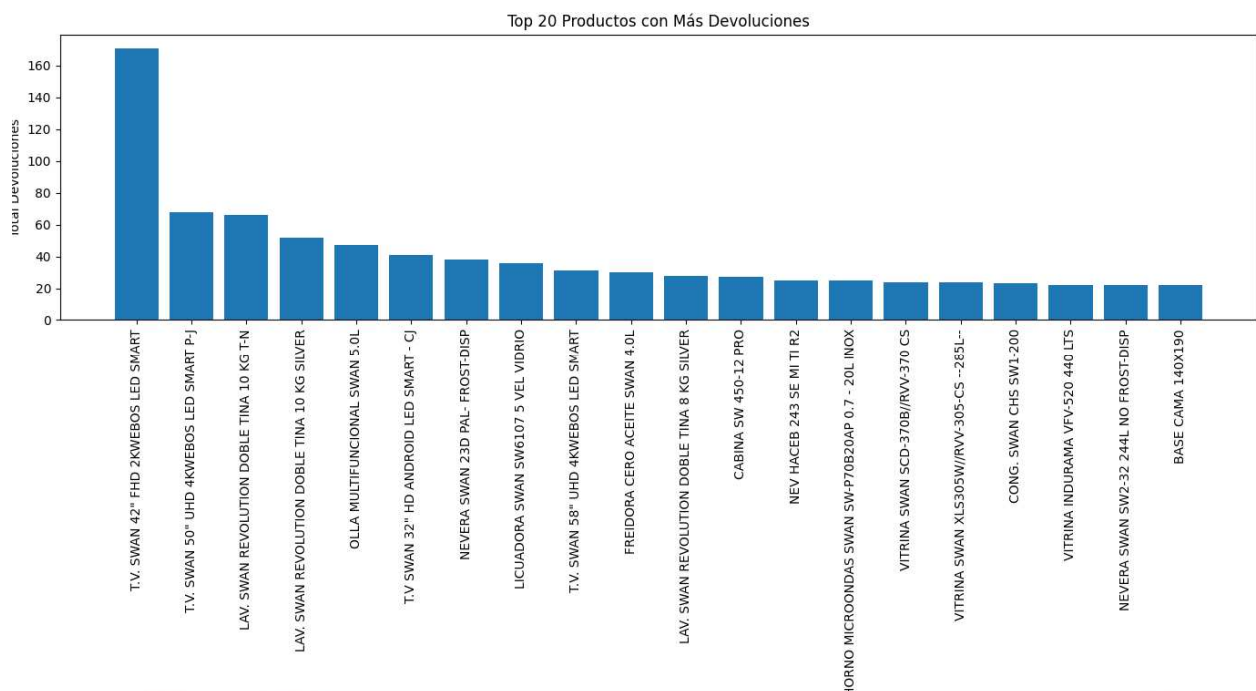


Figura 5: Gráfica del comportamiento de las devoluciones por Marca – Top 20

5. Modelo de Sugerido de Compras

El Notebook **04_modelo_compras** constituye el producto final del pipeline. Integra la información de ventas y devoluciones para calcular la **Demanda Real Neta**.

Lógica y Código del Modelo:

Para evitar el desabastecimiento, se aplica una política de inventarios que incluye un **Stock de Seguridad del 20%**.

$$\text{Sugerido de Compra} = (\text{Ventas Unidades} - \text{Devoluciones Unidades}) * 1.20$$

```
from pyspark.sql.functions import (
    col,
    upper,
    trim,
    regexp_replace,
    expr,
    when,
    abs,
    sum,
    avg,
    round,
    first
)

# =====
# LEER DATOS DESDE SILVER
# =====
df = spark.table("workspace.silver.movcomercial")

# =====
# NORMALIZAR CLASE
# =====
df = df.withColumn(
    "CLASE",
    upper(trim(col("CLASE")))
```

```

)

# =====
# LIMPIAR Y CONVERTIR CANTIDAD
# =====
df = df.withColumn(
    "CANTIDAD_TMP",
    regexp_replace(col("CANTIDAD"), r"\.", "")
)

df = df.withColumn(
    "CANTIDAD_TMP",
    regexp_replace(col("CANTIDAD_TMP"), ",", ".")
)

df = df.withColumn(
    "CANTIDAD_TMP",
    expr("try_cast(CANTIDAD_TMP as double)")
)

# =====
# CREAR VENTA NETA
# FV00 = venta
# DV00 = devolución (ya viene negativa)
# =====
df = df.withColumn(
    "VENTA_NETA_UNIDADES",
    when(
        col("CLASE") == "FV00",
        col("CANTIDAD_TMP")
    ).when(
        col("CLASE") == "DV00",
        col("CANTIDAD_TMP")
    ).otherwise(0)
)

# =====
# AGRUPAR POR MES Y PRODUCTO
# =====
ventas_mensuales = df.groupBy(
    "ANIO",
    "MES",
    "PRODUCTO"
).agg(

    first("PRODUCTONO")
    .alias("NOMBRE_PRODUCTO"),

    # Ventas netas en unidades
    round(

```

```

        sum("VENTA_NETA_UNIDADES"),
        2
    ).alias("VENTAS_NETAS_UNIDADES"),

    # Ventas brutas
    round(
        sum(
            when(
                col("CLASE") == "FV00",
                col("PARCIAL")
            ).otherwise(0)
        ),
        2
    ).alias("VENTAS_BRUTAS"),

    # Devoluciones
    round(
        abs(
            sum(
                when(
                    col("CLASE") == "DV00",
                    col("PARCIAL")
                ).otherwise(0)
            )
        ),
        2
    ).alias("DEVOLUCIONES")

)

# =====
# PROMEDIO HISTÓRICO POR PRODUCTO
# =====
promedio_producto = ventas_mensuales.groupBy(
    "PRODUCTO"
).agg(

    first("NOMBRE_PRODUCTO")
    .alias("NOMBRE_PRODUCTO"),

    round(
        avg("VENTAS_NETAS_UNIDADES"),
        2
    ).alias("PROMEDIO_MENSUAL")

)

# =====
# CALCULAR STOCK DE SEGURIDAD
# =====

```

```

modelo_compras = promedio_producto.withColumn(
    "STOCK_SEGURIDAD",
    round(
        col("PROMEDIO_MENSUAL") * 0.20,
        2
    )
)

# =====
# SUGERIDO DE COMPRA
# =====
modelo_compras = modelo_compras.withColumn(
    "SUGERIDO_COMPRA",
    round(
        col("PROMEDIO_MENSUAL") +
        col("STOCK_SEGURIDAD"),
        0
    )
)

# =====
# ORDENAR RESULTADO
# =====
modelo_compras = modelo_compras.orderBy(
    col("SUGERIDO_COMPRA").desc()
)

# =====
# MOSTRAR RESULTADO
# =====
display(modelo_compras)

# =====
# CREAR ESQUEMA GOLD
# =====
spark.sql("""
CREATE SCHEMA IF NOT EXISTS workspace.gold
""")

# =====
# GUARDAR MODELO EN GOLD
# =====

modelo_compras.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("workspace.gold.sugerido_compras")

print("Tabla workspace.gold.sugerido_compras creada correctamente")

```

```

# =====
# VALIDAR TABLA FINAL
# =====
df_gold = spark.table(
    "workspace.gold.sugerido_compras"
)

display(df_gold)import matplotlib.pyplot as plt

# =====
# LEER MODELO DESDE GOLD
# =====
df = spark.table("workspace.gold.sugerido_compras")

# =====
# TOP 20 PRODUCTOS
# =====
top20 = df.orderBy(
    col("SUGERIDO_COMPRA").desc()
).limit(20)

# =====
# CONVERTIR A PANDAS
# =====
pdf = top20.toPandas()

# =====
# GRAFICAR TORTA
# =====
plt.figure(figsize=(12,12))

plt.pie(
    pdf["SUGERIDO_COMPRA"],
    labels=pdf["NOMBRE_PRODUCTO"],
    autopct='%1.1f%%'
)

plt.title("Top 20 Productos - Sugerido de Compras")
plt.tight_layout()
plt.show()

```

	A _C PRODUCTO	A _C NOMBRE_PRODUCTO	1.2 PROMEDIO_MENSUAL	1.2 STOCK_SEGURIDAD	1.2 SUGERIDO_COMPRA
1	8012131	T.V SWAN 32 HD ANDROID LED SMART - EL	3290	658	3948
2	8012132	T.V. SWAN 40" FHD ANDROID LED SMART - CJ	3140	628	3768
3	101225	LAV. HACEB 7 KG SA BL	2814.17	562.83	3377
4	340000	INTERESES POR MORA GRAVADO	2357.14	471.43	2829
5	8012127	T.V SWAN 32" HD ANDROID LED SMART - CJ	2328	465.6	2794
6	8012134	T.V SWAN 55" UHD 4KWEBOS LED SMART- CJ	1865	373	2238
7	251280	CONG. SWAN CHS SW1-200	1671.67	334.33	2006
8	8012133	T.V SWAN 43" FHD WEBOS LED SMART - CJ	1545	309	1854
9	3412141	VENT. SWAN 18P 3 EN 1 NE	1350.83	270.17	1621
10	8012128	T.V SWAN 43 FHD 2KWEBOS LED SMART	1318.57	263.71	1582
11	8012130	T.V SWAN 50" UHD 4KWEBOS LED SMART- EL	1267.5	253.5	1521
12	3011262	NEV HACEB 243 SE MI TI R2	1204.17	240.83	1445
13	101226	LAV. HACEB 13 KG SA BL	1151	230.2	1381
14	251281	CONG. SWAN CHS SW1-101	1130.91	226.18	1357
15	8080351	T.V LG 43LM6370PDB.AWC	1121.67	224.33	1346

↓ 1746 rows | 521s runtime

Figura 6: Visualización de los datos del modelo en tabla spark en Gold.

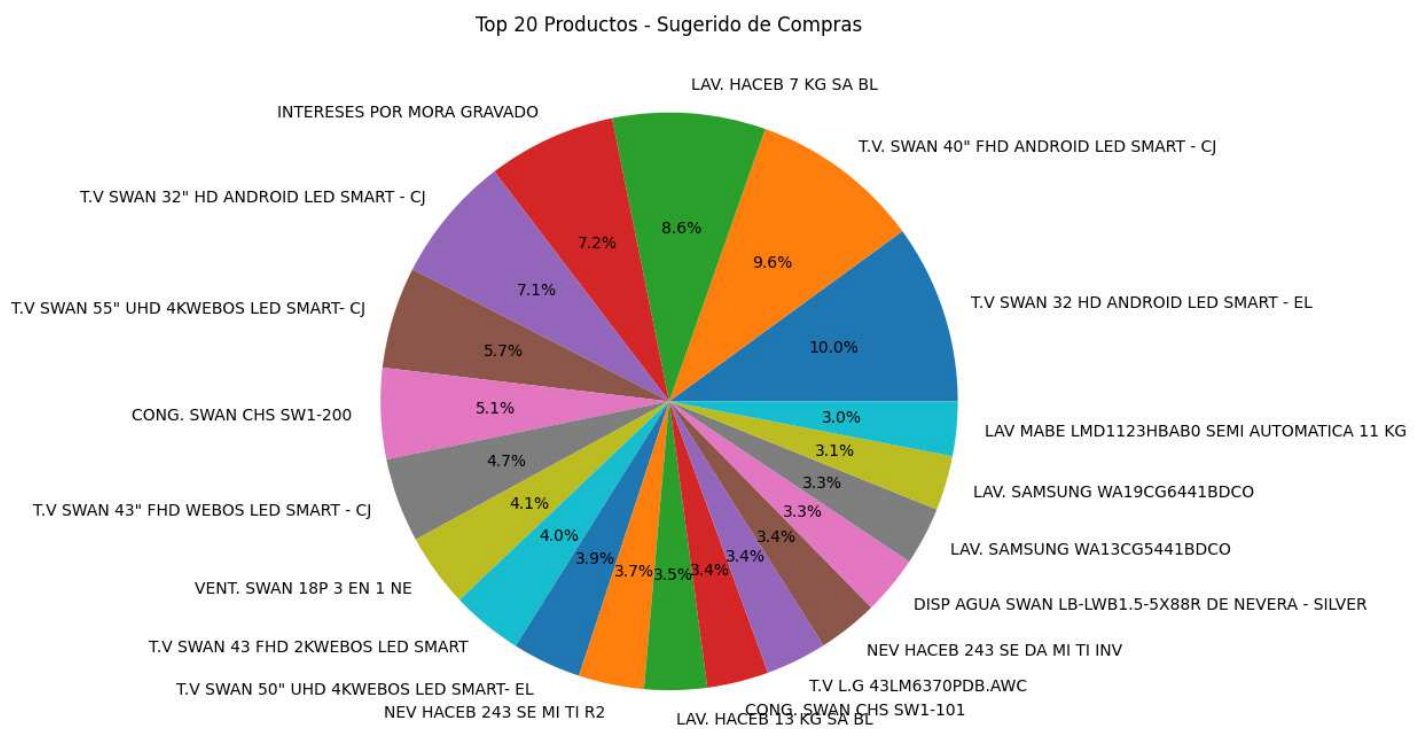


Figura 7: Visualización del modelo gráficamente Top 20 Sugerido en Matplotlib

6. Orquestación y Automatización

Para garantizar que el negocio cuente con información actualizada diariamente, el pipeline ha sido automatizado mediante **Databricks Workflows**:

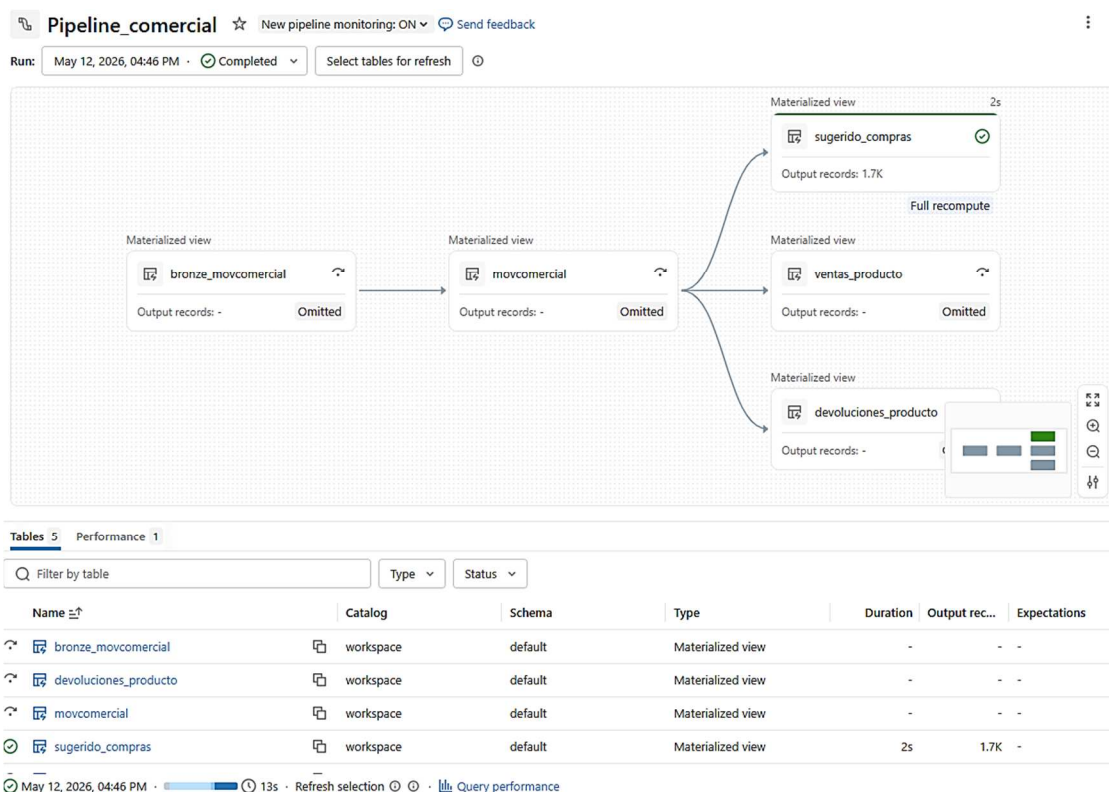
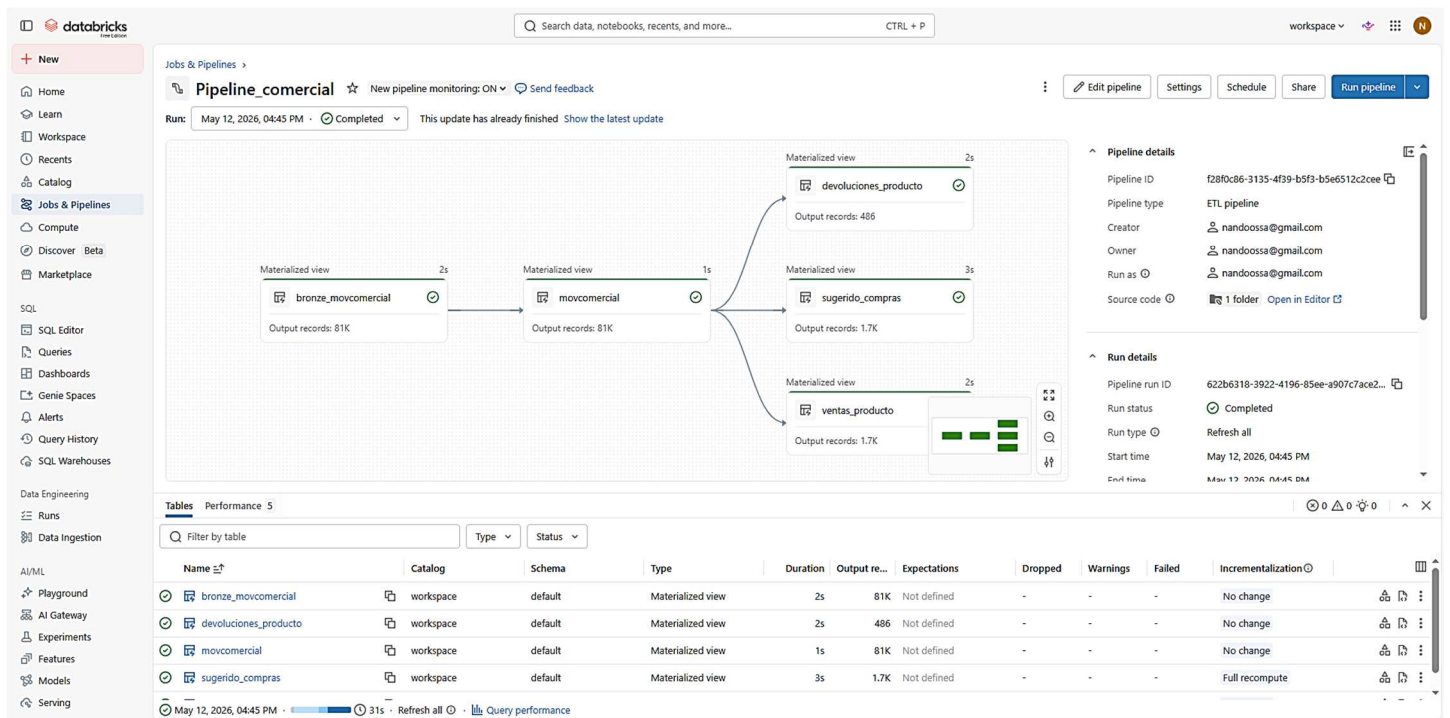
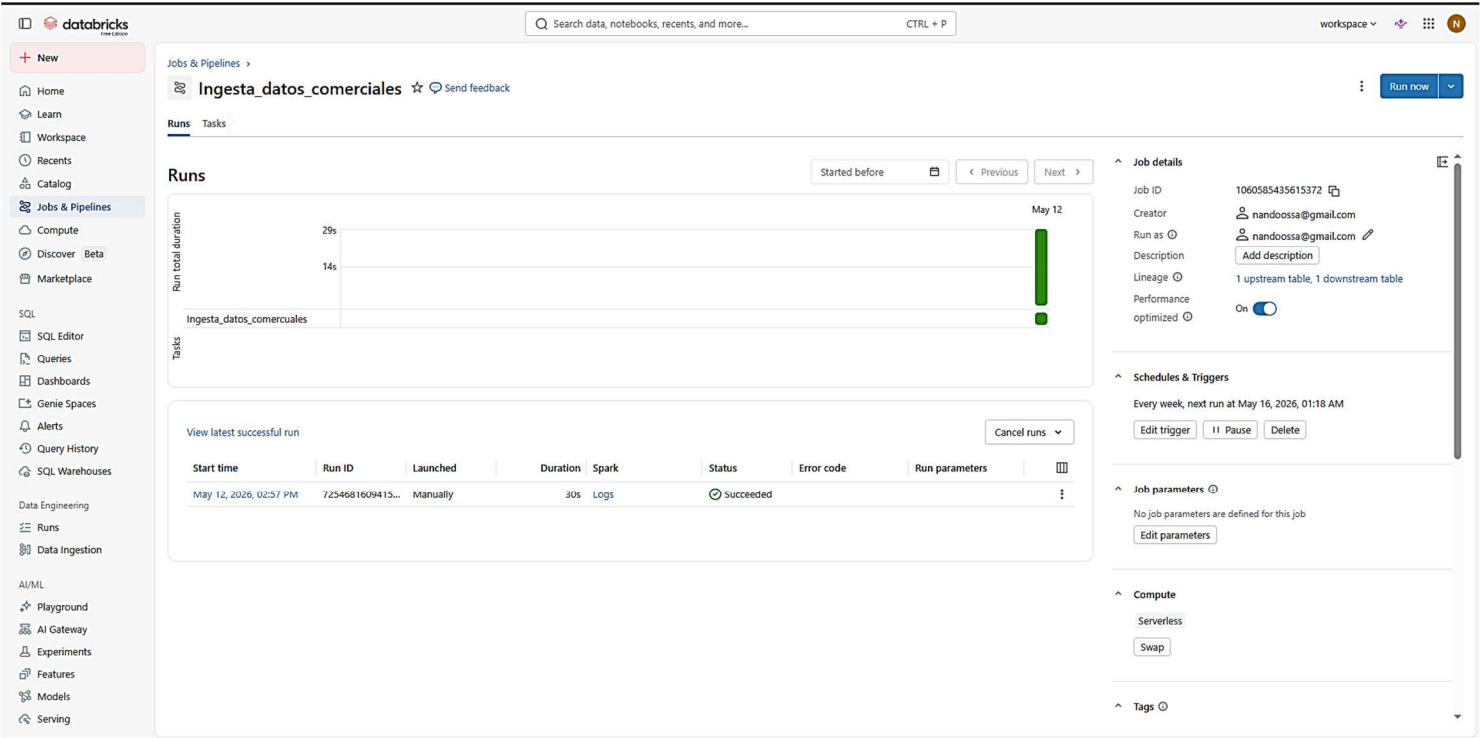


Figura 8 y 9: Linaje de Datos en Delta Live Tables

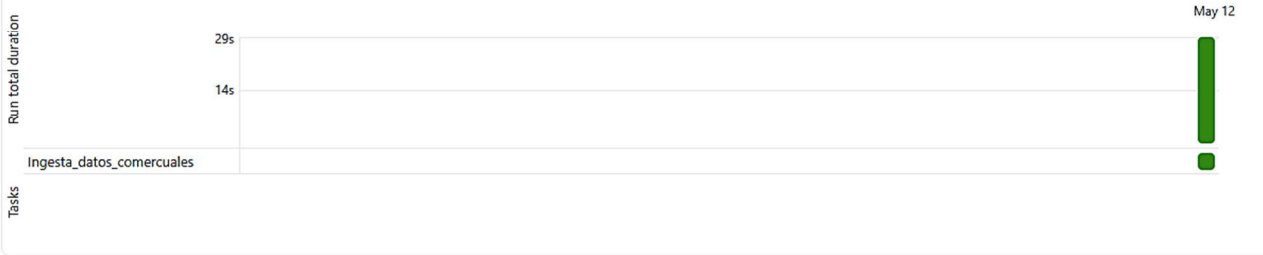


Jobs & Pipelines >
Ingesta_datos_comerciales ☆ Send feedback

Runs Tasks

Runs

Started before < Previous Next >



View latest successful run

Cancel runs ▼

Start time	Run ID	Launched	Duration	Spark	Status	Error code	Run parameters	
May 12, 2026, 02:57 PM	7254681609415...	Manually	30s	Logs	Succeeded			

Figura 10 y 11: Configuración del Job de Ingesta y Procesamiento

7. Conclusiones

La ejecución de este proyecto representa un avance significativo en la madurez analítica de la organización, logrando la transición de una gestión de datos artesanal hacia una infraestructura de datos de clase empresarial

- Transformación Digital y Operativa

La implementación de la arquitectura Medallion permitió convertir procesos manuales y vulnerables en un pipeline automatizado, escalable y robusto. Se logró centralizar la operación en Databricks, garantizando que el flujo de datos sea eficiente desde la ingesta hasta el análisis final.

- Calidad y Gobernanza del Dato

Mediante el uso de Unity Catalog, el proyecto asegura la integridad y seguridad de la información. La estandarización en la capa Silver (limpieza con Regex y normalización) garantiza que las decisiones de negocio se basen en una "única fuente de verdad" confiable y libre de inconsistencias.

- Optimización Estratégica del Inventario

El Modelo de Sugerido de Compras actúa como un motor de inteligencia comercial. Al integrar un margen de seguridad del 20% y deducir las devoluciones de las ventas reales, se optimizan los recursos financieros, reduciendo el riesgo de quiebres de stock y mejorando la rentabilidad operativa.

- Automatización y Visibilidad Analítica

Gracias a la orquestación de Jobs en Databricks, el sistema opera de forma autónoma ante nuevas cargas de datos. Este ecosistema se complementa con visualizaciones críticas:

- Linaje del Pipeline: Trazabilidad total del dato.

- Ventas vs. Devoluciones: Claridad sobre la demanda neta.

- Gráfico de Sugerido: Visión estratégica de la distribución del inventario.

8. Documentación:

| Repositorio y Readme:

El repositorio contiene la estructura completa del pipeline de datos, dividida en notebooks de Databricks y recursos visuales:

Notebooks de Proceso (.ipynb):

- 01_ingestar_datos_bronze.ipynb: Contiene la lógica para la creación de esquemas y la ingesta inmutable de los datos crudos.
- 02_silver.ipynb: Archivo dedicado a la limpieza profunda, normalización de textos y corrección de formatos numéricos mediante Regex.
- 03_gold.ipynb: Implementa las agregaciones de negocio para ventas y devoluciones por producto.
- 04_modelo_compras.ipynb: El módulo final que calcula la demanda neta y el sugerido de compras con el margen de seguridad.

Repositorio Github: https://github.com/nandoossa/Trabajo_Final_BD

Recursos Visuales y Documentación (Imágenes):

JOB_INGESTA.png: Captura de la orquestación y programación del flujo de datos en Databricks Workflows.

MAPA DEFINITIVO MEDALLION COMERCIAL.png: Diagrama de la arquitectura de extremo a extremo (End-to-End).

PIPELINE.png: Visualización del linaje de datos y las dependencias entre tablas.

Documentación Adicional:

Documentación Pipeline Medallion Databricks, llamado README.pdf Un archivo que sirve como manual técnico detallado del proyecto.

nandoossa Imagen	902b3cf · 1 minute ago	7 Commits
pipelines	ok	yesterday
01_ingestar_datos_bronze.ipynb	ok	yesterday
02_silver.ipynb	ok	yesterday
03_gold.ipynb	ok	yesterday
04_modelo_compras.ipynb	ok	yesterday
JOB_INGESTA.png	Imagenes	8 minutes ago
MEDALLION MAP.png	Ok	1 minute ago
PIPELINE.png	Imagen	1 minute ago
README.md	Ok	2 minutes ago

README

Arquitectura ELT en Databricks para análisis comercial y abastecimiento de inventario

Figura 12: Entorno Repositorio Github

Raúl Fernando Ossa Ramírez

Jennifer Alejandra López Sánchez

Universidad Autónoma Latinoamericana UNAULA

Medellín – Colombia

2.026